

Diamond Price Analysis Project

Name	Mayur Gadhave
Roll Number	_____
Project Title	Diamond Price Analysis Using Python
Dataset	Diamonds dataset
GitHub Link	Paste your GitHub repository link here before final submission

1. Problem Statement

I am analyzing the Diamonds dataset to understand which features are most strongly related to diamond price. This can help a jewelry shop or buyer make better pricing decisions by identifying how size, quality, and dimensions affect the final cost.

Dataset name: **Diamonds**. Source: **Seaborn sample dataset repository**. Original size: **53,940 rows and 10 columns**. The columns are carat, cut, color, clarity, depth, table, price, x, y, and z.

Column	Meaning
carat	Weight of the diamond
cut	Quality of cut
color	Color grade
clarity	Clarity grade
depth	Total depth percentage
table	Top width relative to widest point
price	Price in US dollars
x, y, z	Dimensions of the diamond in millimeters

Imports - code screenshot

Code Cell

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

All libraries were imported in one code cell as required.

2. Data Loading and Basic Information

Code screenshot

Code Cell

```
df = pd.read_csv("diamonds.csv")
df.head()
print("Shape:", df.shape)
print(df.dtypes)
```

Output screenshot

Output: First 5 Rows

carat	cut	color	clarity	depth	table	price	x	y	z
0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

Output: Shape and Data Types

Shape of dataset: (53940, 10)

Data types:

carat	float64
cut	object
color	object
clarity	object
depth	float64
table	float64
price	int64
x	float64
y	float64
z	float64

The dataset satisfies the assignment rule because it has more than 1,000 rows, contains 10 columns, and includes both numeric and categorical data.

3. Data Cleaning

Code screenshot

Code Cell

```
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# No missing values were found, so no filling was needed
df = df.drop_duplicates()
print("New shape after removing duplicates:", df.shape)
```

Output screenshot

Output: Missing Values and Duplicates

Missing values:

carat	0
cut	0
color	0
clarity	0
depth	0
table	0
price	0
x	0
y	0
z	0

Duplicate rows found: 146
Duplicate rows after removal: 0

No missing values were found, so no filling was required. Duplicate rows were removed because repeated records can make statistics and model results less reliable.

4. Descriptive Statistics

Code screenshot

Code Cell

```
def descriptive_stats(series):
    return {
        "Mean": series.mean(),
        "Median": series.median(),
        "Mode": series.mode()[0],
        "Standard Deviation": series.std(),
        "Variance": series.var(),
        "Range": series.max() - series.min(),
        "Mid-range": (series.max() + series.min()) / 2
    }

carat_stats = descriptive_stats(df["carat"])
price_stats = descriptive_stats(df["price"])

stats_df = pd.DataFrame({"carat": carat_stats, "price": price_stats})
print(stats_df)
```

Output screenshot

Output: Descriptive Statistics

	carat	price
Mean	0.80	3932.80
Median	0.70	2401.00
Mode	0.30	605.00
Std Dev	0.47	3989.44
Variance	0.22	15915629.42
Range	4.81	18497.00
Mid-range	2.60	9574.50

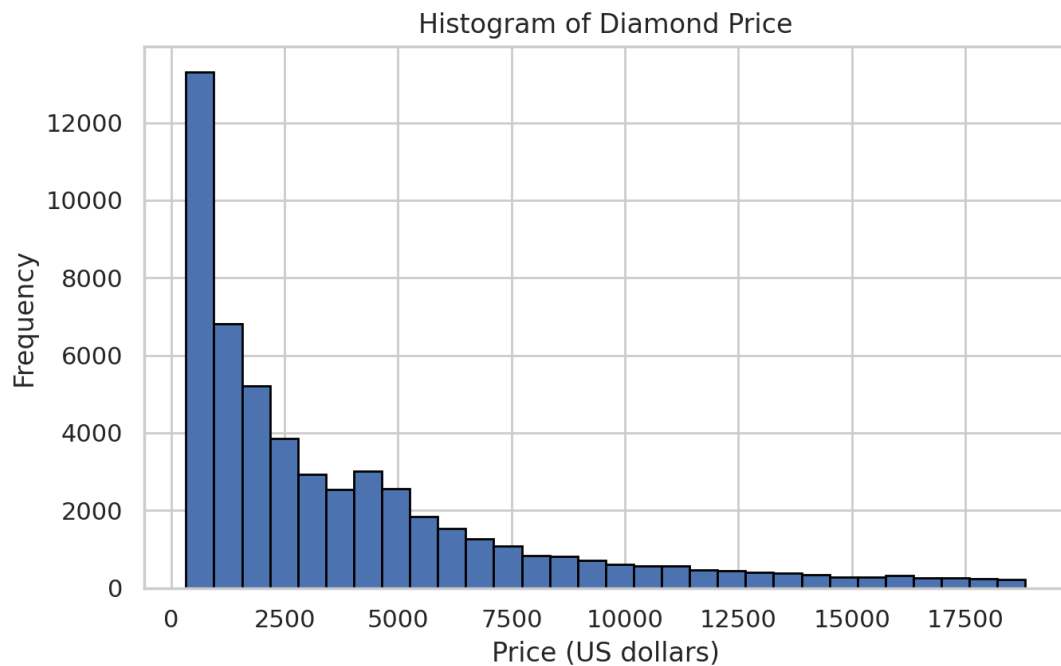
For the numeric columns **carat** and **price**, I calculated mean, median, mode, standard deviation, variance, range, and mid-range as required. Price has much larger spread than carat, which shows strong variation in diamond prices.

5. Visualizations

Histogram

Code Cell

```
plt.figure(figsize=(7,4))
plt.hist(df["price"], bins=30, edgecolor="black")
plt.title("Histogram of Diamond Price")
plt.xlabel("Price (US dollars)")
plt.ylabel("Frequency")
plt.show()
```

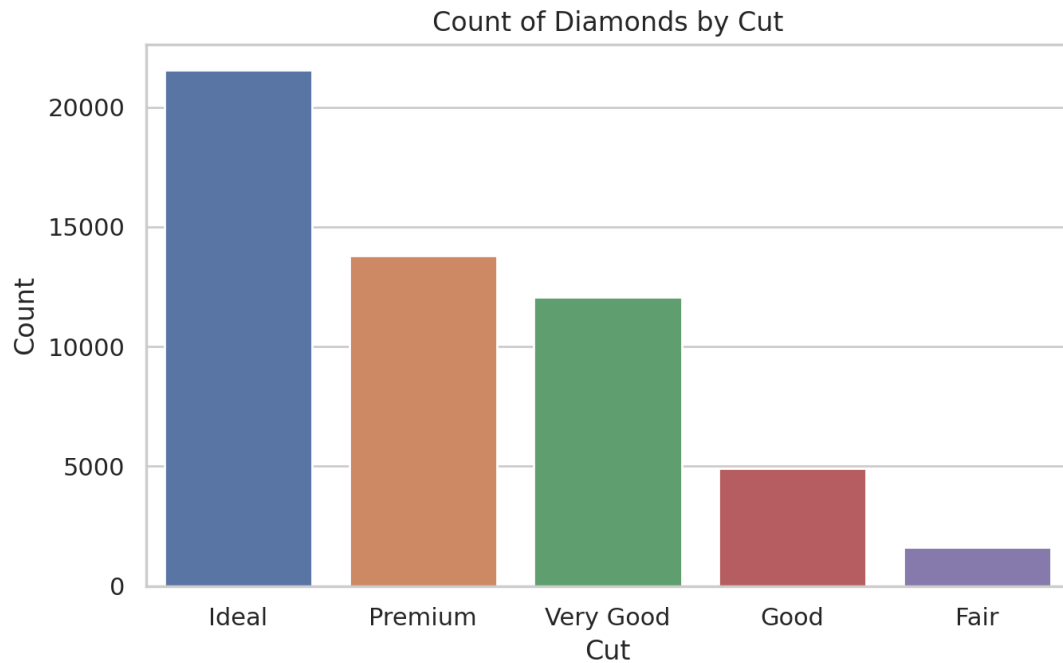


Interpretation: Most diamonds are concentrated in the lower price range, while fewer diamonds appear at very high prices. This shows that the price distribution is right-skewed.

Bar Chart / Countplot

Code Cell

```
plt.figure(figsize=(7,4))
sns.countplot(data=df, x="cut", order=df["cut"].value_counts().index)
plt.title("Count of Diamonds by Cut")
plt.xlabel("Cut")
plt.ylabel("Count")
plt.show()
```

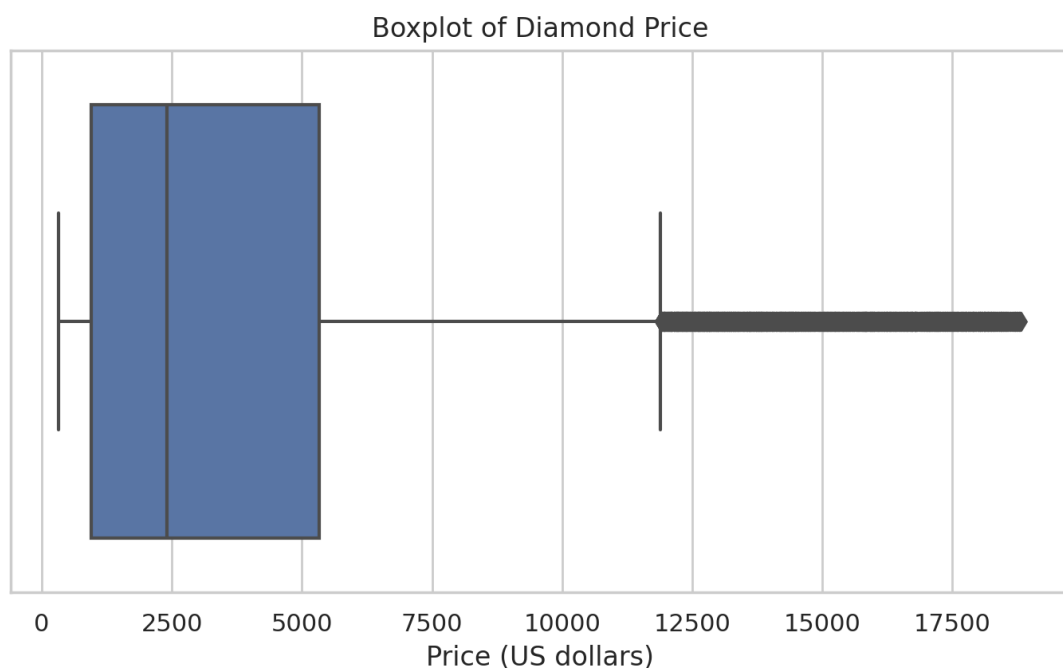


Interpretation: The countplot shows that Ideal cut diamonds are the most common category in this dataset, while Fair cut diamonds are the least common.

Boxplot

Code Cell

```
plt.figure(figsize=(7,4))
sns.boxplot(x=df["price"])
plt.title("Boxplot of Diamond Price")
plt.xlabel("Price (US dollars)")
plt.show()
```

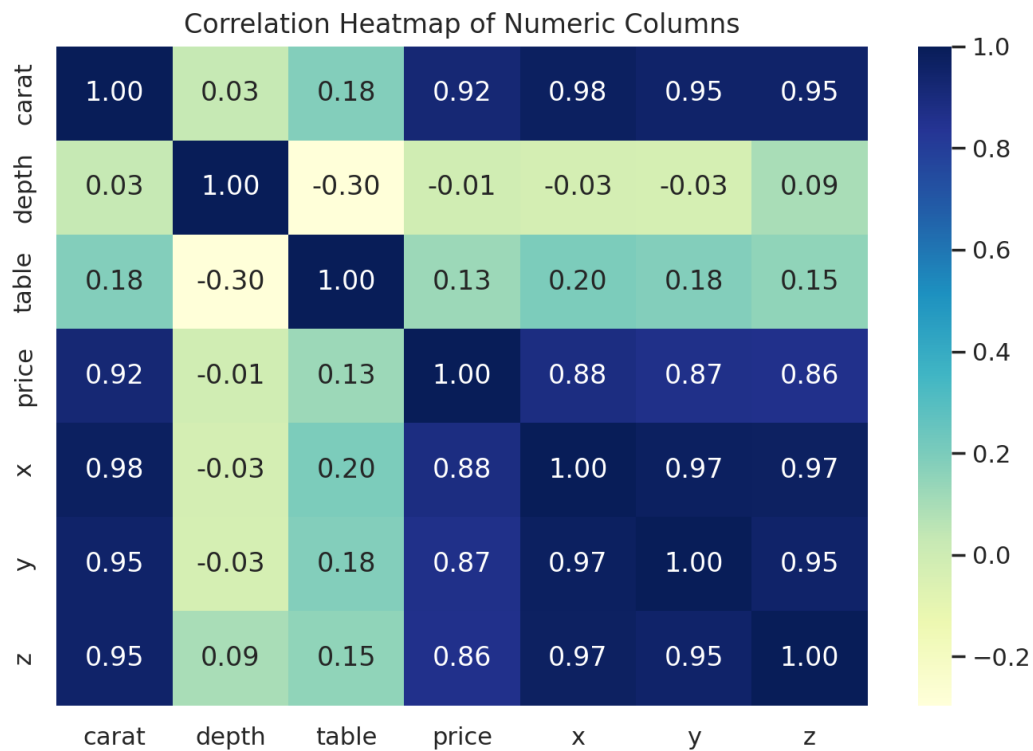


Interpretation: The boxplot shows a wide spread in price and several outliers on the higher side, meaning a small group of diamonds are much more expensive than typical diamonds.

Correlation Heatmap

Code Cell

```
plt.figure(figsize=(7,5))
corr = df.select_dtypes(include="number").corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="YlGnBu")
plt.title("Correlation Heatmap of Numeric Columns")
plt.show()
```



Interpretation: The heatmap shows strong positive correlation between price and carat, as well as with x, y, and z dimensions. Larger diamonds usually cost more.

6. Simple Prediction

Code screenshot

Code Cell

```
X = df.drop("price", axis=1)
y = df["price"]

categorical_cols = ["cut", "color", "clarity"]
numeric_cols = ["carat", "depth", "table", "x", "y", "z"]

preprocess = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
    ("num", "passthrough", numeric_cols)
])

model = Pipeline([
    ("preprocess", preprocess),
    ("regressor", LinearRegression())
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model.fit(X_train, y_train)
pred = model.predict(X_test)

print("R-squared:", r2_score(y_test, pred))
print("MAE:", mean_absolute_error(y_test, pred))
```

Output screenshot

Output: Model Results

Train-test split: 80:20
Model: Linear Regression

R-squared: 0.9189
MAE: 737.15
RMSE: 1135.21

Interpretation:
The model explains about 91.9% of the variation in price.
On average, predictions differ from actual prices by about \$737.

This project uses one-hot encoding for categorical columns and an 80:20 train-test split. A Linear Regression model was then trained to predict diamond price.

Result in simple English: the model explains about **91.9%** of the variation in price, which means it performs well for a simple baseline model. On average, the prediction error is about **\$737**, so the model is useful for estimation but not perfectly exact.

7. Insights and Recommendations

Finding 1: The histogram shows that most diamonds have lower prices and only a smaller number are very expensive.

Finding 2: The boxplot shows several high-price outliers, which means the dataset contains premium diamonds far above the usual price range.

Finding 3: The heatmap shows that carat has a strong positive relationship with price, so heavier diamonds are usually costlier.

Finding 4: The countplot shows that Ideal cut diamonds appear most often, which may affect how common certain quality categories are in the market.

Recommendation 1: A jewelry business should separate low-price and high-price diamonds while planning inventory because the prices are not evenly distributed.

Recommendation 2: Buyers should first compare carat and then check cut, color, and clarity, because size has a strong effect on price and quality features further refine the value.

8. Submission Checklist

This package includes the following files for your submission and GitHub repository:

File	Status
Diamond_Price_Analysis_Project.ipynb	Created
diamonds.csv	Included
README.md	Created
Perfect_Final_Diamond_Project.pdf	Created
GitHub repository upload	Do this manually before final submission

Before submitting, add your roll number and upload the CSV, notebook, README, and PDF to GitHub. Then paste the GitHub link into the PDF if your teacher requires it.